

Taller evaluativo del Módulo 1 ConectaBarrio

Guía para construir y desplegar un MVP con Angular

El reto. Una organización social necesita dejar de gestionar solicitudes comunitarias en mensajes dispersos. Usted diseñará una aplicación Angular que permita consultarlas, buscar, abrir el detalle, marcar casos prioritarios, registrar nuevas solicitudes y publicar el MVP para revisión.

Curso Java Full Stack Grupo jueves

Dev Senior Code

No contiene código de solución.

Antes de comenzar

Este taller no está diseñado para medir cuánto código puede memorizar. Está diseñado para comprobar si puede comprender una necesidad, dividirla en responsabilidades, tomar decisiones técnicas y construir una solución que otra persona pueda usar y revisar.

Durante el recorrido encontrará explicaciones, preguntas de criterio, recordatorios del Módulo 1, comandos de terminal, ayudas de depuración y prompts de IA. No encontrará el contenido resuelto de archivos TypeScript, HTML o CSS. La implementación, las decisiones y la defensa del resultado son suyas.

Una idea para acompañar todo el proceso. Si en algún momento siente que “no sabe por dónde empezar”, no intente resolver toda la aplicación. Vuelva al último control de avance, lea la responsabilidad del archivo actual y complete una sola evidencia observable.

Avanzar en software casi siempre significa transformar un problema grande en decisiones pequeñas que pueden comprobarse.

Ficha del taller

Elemento	Definición
Producto	ConectaBarrio: MVP web para consultar y registrar solicitudes comunitarias.
Evaluación principal	Módulo 1: Angular moderno, arquitectura frontend, routing, Signals, comunicación entre componentes y consumo de APIs.
Modalidad sugerida	Trabajo individual. La instructora puede autorizar parejas; cada integrante debe defender decisiones y contribuciones.
Carga orientativa	12 a 16 horas de trabajo autónomo, distribuidas por hitos. Es una referencia, no una carrera contra el reloj.
Tecnologías	Angular 22 con componentes standalone, TypeScript, HTML, CSS, Angular Router, Signals, HttpClient, Git y una plataforma de despliegue estático.
API de práctica	JSONPlaceholder: recursos posts y users. El POST es simulado y no persiste después de recargar.
Resultado	Repositorio versionado, README, aplicación compilable, URL desplegada, evidencias y bitácora de uso de IA.

Cómo usar este documento

- 1. Lea primero el problema de negocio y el resultado observable del hito.
- 2. Responda la pausa de criterio antes de abrir una IA o buscar una receta.
- 3. Identifique el archivo responsable. No programe “donde funcione”; programe donde corresponda.
- 4. Implemente una porción pequeña y compruébela en el navegador o en la consola.
- 5. Complete el control de avance y guarde evidencia.
- 6. Realice un commit con una intención clara.
- 7. Solo entonces pase al siguiente hito.

Acuerdo de uso responsable de IA

La IA está permitida como copiloto. Puede ayudarle a formular preguntas, revisar una decisión, interpretar un error, proponer una matriz de pruebas o detectar riesgos de accesibilidad. No debe reemplazar su análisis ni producir una solución que usted no pueda explicar.

Uso aceptable	Uso que invalida el aprendizaje
Pedir que compare dos arquitecturas y explique sus consecuencias.	Pedir “hazme toda la aplicación” y copiar la respuesta.
Compartir un error puntual y pedir hipótesis ordenadas para investigarlo.	Pegar archivos completos generados sin leer ni probar.
Solicitar una lista de casos de prueba o una revisión de accesibilidad.	Aceptar una API antigua de Angular solo porque compila.
Pedir preguntas para defender una decisión propia.	Ocultar el uso de IA o atribuirse decisiones que no comprende.

Bitácora obligatoria de IA. Registre al menos tres interacciones: problema que intentaba resolver, prompt utilizado, respuesta recibida, qué aceptó o rechazó, cómo lo comprobó y qué aprendió. Una respuesta de IA no es evidencia; la verificación sí.

Mapa del documento

Bloque	Contenido
Fundamentos	Propósito, caso empresarial, alcance funcional, API y arquitectura propuesta.
Construcción guiada	Partes 0 a 16, desde el análisis y la creación del proyecto hasta el despliegue y la sustentación.
Evaluación	Entregables, evidencias, rúbrica de 100 puntos y lista de verificación final.
Apoyo	Prompts de IA, diagnóstico de errores, glosario, plantilla de evidencias y fuentes de consulta.

1. Qué se está evaluando

El taller retoma las cuatro clases del Módulo 1 y las integra en un solo producto. Cada competencia aparece en una necesidad real del caso; ninguna actividad es un ejercicio aislado.

Clase del módulo	Conceptos retomados	Dónde se evidencian
Clase 1 · Fundamentos de Angular	Frontend, HTML, CSS, TypeScript, CLI, componentes standalone, data binding, interpolación, property binding, @for, @if, @Input y arquitectura de carpetas.	Creación del proyecto, sistema visual, tarjeta reutilizable, listado y estados de interfaz.
Clase 2 · Routing y navegación	SPA, páginas frente a componentes, routerLink, router-outlet, parámetros, withComponentInputBinding, lazy loading, rutas profundas y sistema de diseño.	Cascarón, mapa de rutas, navegación, detalle por id y comprobación después del despliegue.
Clase 3 · Estado y comunicación	signal, computed, estado local/compartido, Input, output, flujo unidireccional, servicios, inject, inmutabilidad, accesibilidad y event bubbling.	Búsqueda, casos prioritarios, contador global, tarjeta que emite eventos y fuente única de verdad.
Clase 4 · APIs y backend	HttpClient, Observable, subscribe, GET, POST, contrato JSON, carga, error, validación, servicios HTTP y actualización del Signal.	Carga de solicitudes y líderes, registro, estados asíncronos y manejo de limitaciones de la API.

Resultados de aprendizaje

- Traducir una necesidad de negocio en historias de usuario y criterios verificables.
- Crear y organizar una aplicación Angular moderna con componentes standalone.
- Separar responsabilidades entre páginas, componentes reutilizables, modelos y servicios.
- Construir navegación SPA con rutas estáticas, ruta parametrizada y carga diferida.
- Modelar el estado con Signals, valores derivados con computed y actualización inmutable.
- Comunicar componentes con datos hacia abajo y eventos hacia arriba.
- Consumir recursos REST con HttpClient y representar carga, vacío, éxito y error.
- Enviar un POST respetando el contrato del servidor y actualizar la interfaz sin recargar.
- Diseñar una interfaz responsive, navegable con teclado y comprensible en estados adversos.
- Versionar, documentar, compilar y desplegar un producto revisable.

Alcance y límites

Dentro del alcance. Frontend Angular funcional; navegación; componentes; estado; consulta y creación simulada por API; manejo de errores; documentación; Git; compilación y despliegue.

Fuera del alcance de este módulo. Autenticación, JWT, guards, interceptores, roles, Spring Security, creación o modificación del backend, base de datos propia y pruebas automatizadas. Estos temas pertenecen a módulos posteriores. No adelante seguridad “por impresionar”. Una solución profesional también sabe respetar el alcance.

2. Caso de negocio ConectaBarrio

La historia

RedManos es una organización social que acompaña a líderes comunitarios de varios barrios. Recibe solicitudes para jornadas de limpieza, apoyo escolar, recuperación de espacios, donaciones y acompañamiento a personas mayores. Hoy cada solicitud llega por un canal distinto: mensajes, llamadas, correos y notas personales.

La coordinadora, Valeria, tarda demasiado en encontrar información, no puede abrir un caso desde un enlace directo y pierde de vista cuáles solicitudes necesitan seguimiento prioritario. Además, cuando presenta resultados a aliados, no tiene un producto que pueda abrirse desde cualquier dispositivo.

RedManos no necesita todavía un sistema definitivo. Necesita un MVP: una versión pequeña pero utilizable que permita validar el flujo, demostrar la arquitectura y preparar la futura integración con un backend propio.

Pregunta de negocio

Piense antes de continuar. Si una coordinadora tiene cien solicitudes en mensajes dispersos, ¿qué es más urgente: agregar muchas funciones o asegurar que pueda encontrar, comprender, priorizar y registrar un caso sin perder contexto? Orientación conceptual: un MVP no es una aplicación incompleta al azar. Es la porción mínima que resuelve un flujo valioso de principio a fin y permite aprender con evidencia.

Usuarios y necesidades

Actor	Necesidad	Resultado esperado
Coordinadora de operación	Ver el panorama, buscar solicitudes, abrir detalle, marcar prioridad y registrar nuevos casos.	Decide qué atender sin revisar múltiples canales.
Líder comunitario	Aparecer asociado a una solicitud con datos de contacto básicos.	La organización sabe quién reporta o acompaña el caso.
Aliado o evaluador	Abrir el MVP desde una URL y recorrerlo sin instalar nada.	Puede revisar valor, estabilidad y claridad del producto.
Equipo de desarrollo futuro	Entender la estructura y reemplazar la API simulada por un backend real.	La arquitectura permite crecer sin reescribir toda la interfaz.

Objetivo del producto

Construir una aplicación web responsive que consulte solicitudes comunitarias y líderes desde una API REST, permita navegar al detalle de cada solicitud, buscar y marcar prioridades, registre una nueva solicitud mediante POST, comunique con claridad los estados de carga/error y esté disponible en una URL pública.

Historias de usuario obligatorias

ID	Historia
HU-01	Como coordinadora, quiero ver una página de inicio con el propósito del sistema y un resumen operativo para orientarme.

ID	Historia
HU-02	Como coordinadora, quiero ver las solicitudes en tarjetas legibles para escanear el trabajo pendiente.
HU-03	Como coordinadora, quiero buscar por título o contenido para encontrar un caso sin recorrer toda la lista.
HU-04	Como coordinadora, quiero abrir una solicitud mediante una URL con id para consultar su detalle y compartir el enlace.
HU-05	Como coordinadora, quiero marcar o desmarcar una solicitud como prioritaria y ver un contador global.
HU-06	Como coordinadora, quiero consultar líderes comunitarios para comprender quién puede estar relacionado con los casos.
HU-07	Como coordinadora, quiero registrar una solicitud con líder, título y descripción para completar el flujo de escritura.
HU-08	Como usuaria, quiero saber si el sistema está cargando, no encontró datos o falló para no confundir silencio con funcionamiento.
HU-09	Como evaluador, quiero abrir el MVP desplegado y refrescar una ruta interna sin recibir un 404 del servidor.

Definición de “terminado”

- ☐ La aplicación inicia sin errores y la compilación de producción finaliza correctamente.
- ☐ Todas las rutas obligatorias funcionan por navegación y por acceso directo.
- ☐ Los datos remotos se muestran desde servicios, no desde arreglos escritos dentro de las páginas.
- ☐ La lista representa carga, error, vacío y datos disponibles como estados diferentes.
- ☐ El buscador y las prioridades usan Signals; los cálculos derivados no se duplican.
- ☐ La tarjeta reutilizable recibe datos y emite la intención de cambiar prioridad.
- ☐ El formulario valida antes de enviar y el POST exitoso se refleja en la sesión actual.
- ☐ La interfaz es usable en celular y con teclado.
- ☐ README, repositorio, URL desplegada y bitácora de IA permiten revisar el proceso.
- ☐ El estudiante puede explicar el recorrido del dato de extremo a extremo.

3. Contrato de la API de práctica

El MVP utilizará JSONPlaceholder, una API pública de práctica. Los nombres del contrato son deliberadamente genéricos. En la interfaz, el recurso posts se interpretará como “solicitudes” y users como “líderes”. Esta traducción pertenece a la capa de presentación; el servicio debe respetar la forma real del JSON.

Advertencia profesional. JSONPlaceholder simula operaciones. Un POST responde como si hubiera creado el recurso, pero no lo guarda de forma permanente. El nuevo elemento puede vivir en el Signal durante la sesión; al recargar, volverán los datos originales de la API.

El README y la interfaz deben comunicar esta limitación con honestidad. Ocultarla convertiría una demostración en una promesa falsa.

URL base y endpoints

URL base: <https://jsonplaceholder.typicode.com>

Método	Recurso	Uso en ConectaBarrio	Respuesta esperada
GET	/posts	Cargar todas las solicitudes.	Colección de 100 objetos.
GET	/posts/{id}	Consultar una solicitud puntual; puede usarse como contraste, aunque el detalle también puede derivarse del estado ya cargado.	Un objeto o respuesta vacía/no encontrada según el id.
GET	/users	Cargar líderes para consulta y selección.	Colección de 10 usuarios.
GET	/users/{id}	Consultar datos del líder asociado.	Un usuario.
POST	/posts	Simular el registro de una nueva solicitud.	Objeto devuelto con id simulado y estado HTTP de creación.

Contrato solicitud remota

Campo	Tipo	Lectura de negocio	Regla
id	number	Identificador de la solicitud.	Lo entrega la API. No lo inventa el formulario.
userId	number	Identificador del líder relacionado.	Debe coincidir con un usuario existente en el recurso users.
title	string	Asunto breve de la solicitud.	Obligatorio para registrar.
body	string	Descripción y contexto del caso.	Obligatorio para registrar.

Contrato líder remoto

Campo de interés	Tipo	Uso en el MVP
id	number	Relacionar líder y solicitud.
name	string	Nombre visible del líder.
email	string	Dato de contacto.
phone	string	Dato de contacto secundario.
address.city	string	Referencia territorial disponible en la API.
company.name	string	Organización de referencia; puede mostrarse como información secundaria.

Pausa de criterio contrato frente a interfaz

Pregunta. ¿Por qué no conviene llamar barrio al campo address.city dentro de la interfaz TypeScript que representa exactamente la respuesta remota?

Orientación conceptual: una interfaz de transporte describe el contrato externo. Cambiar nombres exige una transformación explícita. Mezclar ambas cosas vuelve más difícil rastrear errores y sustituir la API.

4. Arquitectura esperada

La arquitectura no se evalúa por la cantidad de carpetas. Se evalúa por la claridad de las responsabilidades. Cada pieza debe tener una razón de existir y una frontera comprensible.

Flujo principal del dato

Origen	Responsabilidad	Entrega a
API REST	Responde JSON o error HTTP.	Servicios Angular.
Servicios	Hablan con la API, actualizan Signals y exponen estado de solo lectura.	Páginas y cascarón.
Páginas	Coordinan el caso de uso de una ruta, derivan estado local y componen piezas.	Componentes reutilizables.
Componentes	Presentan una unidad visual, reciben datos y emiten intenciones.	Página padre.
Templates	Declaran lo visible para cada estado y enlazan eventos.	Persona usuaria.

Mapa de rutas obligatorio

Ruta	Página	Propósito	Carga
/	Inicio	Contexto, indicadores y llamadas a la acción.	Directa o diferida, pero justificada.
/solicitudes	Listado	Buscar, recorrer y priorizar solicitudes.	Lazy loading.
/solicitudes/nueva	Nueva solicitud	Validar y enviar el POST.	Lazy loading.
/solicitudes/:id	Detalle	Abrir un caso mediante parámetro de ruta.	Lazy loading.
/lideres	Líderes	Consultar responsables disponibles.	Lazy loading.
** · opcional	No encontrada	Explicar el error y ofrecer regreso seguro.	Lazy loading.

Mapa de archivos y responsabilidades

Ruta o archivo	Qué debe vivir allí	Qué no debe vivir allí
src/app/pages/inicio/	Composición del inicio, indicadores derivados y accesos principales.	Llamadas HTTP repetidas ni tarjetas copiadas.
src/app/pages/solicitudes-listado/	Buscador local, resultado filtrado, composición de tarjetas y estados de lista.	URL de API ni lógica duplicada de prioridades.
src/app/pages/solicitud-detalle/	Lectura del id, selección del caso, estado no encontrado y acciones de detalle.	Una segunda copia de la colección.
src/app/pages/solicitud-nueva/	Formulario, validación previa, envío y retroalimentación.	Construcción manual del id.
src/app/pages/lideres-listado/	Presentación de líderes y sus estados asíncronos.	Peticiones realizadas directamente desde el componente.
src/app/components/solicitud-card/	Presentar una solicitud, recibir prioridad y emitir	Buscar en la API o modificar el

Ruta o archivo	Qué debe vivir allí	Qué no debe vivir allí
	la intención de cambio.	servicio directamente.
src/app/services/solicitudes.service.ts	Contrato de solicitudes, GET/POST, Signals, prioridad y métodos del dominio.	HTML, estilos o decisiones de navegación visual.
src/app/services/lideres.service.ts	GET de líderes y estado remoto asociado.	Lógica de presentación del formulario.
src/app/models/	Interfaces que explican la forma de los datos.	Estado mutable o llamadas HTTP.
src/app/app.routes.ts	Tabla de rutas y carga diferida.	Reglas de negocio.
src/app/app.config.ts	Proveedores globales: router y HttpClient.	Datos de ejemplo.
src/app/app.html y app.ts	Cascarón, navegación, router-outlet y contador global.	El contenido completo de todas las páginas.
src/styles.css	Tokens visuales, normalización y estilos verdaderamente globales.	Estilos específicos de una tarjeta.

Criterio de arquitectura. Una página conoce el caso de uso de su ruta. Un componente reutilizable conoce su presentación. Un servicio conoce el estado y la comunicación remota. Un modelo conoce la forma del dato. Cuando una pieza hace las cuatro cosas, el sistema funciona hoy, pero se vuelve difícil de modificar mañana.

5. Ruta de construcción guiada

La secuencia siguiente está pensada para que cada hito reutilice el anterior. No salte directamente al POST o al despliegue. Primero construya una base comprobable.

Parte 0 Pensar antes de programar

PROPÓSITO

Acordar qué problema se resolverá, qué no se construirá y cómo se reconocerá el éxito. Esta parte evita que el código se convierta en una colección de funciones sin dirección.

Actividad 0.1 Escriba el contrato del producto

En el README cree una sección inicial de producto. No copie la historia completa: sintetízela con sus propias palabras.

- ☐ Problema en máximo cinco líneas.
- ☐ Usuario principal y decisión que necesita tomar.
- ☐ Cinco funciones imprescindibles del MVP.
- ☐ Tres elementos explícitamente fuera de alcance.
- ☐ Riesgo principal de usar una API simulada.
- ☐ Definición personal de terminado.

Actividad 0.2 Dibuje el recorrido

En papel, pizarra o herramienta de diagramación, dibuje: Inicio → Listado → Detalle → Marcar prioridad → Nueva solicitud → Confirmación. Añada también el camino de error y la forma de regresar. No se evalúa la belleza del dibujo; se evalúa si el flujo tiene continuidad.

Pausa de criterio

Pregunta. ¿Qué diferencia hay entre una función y un flujo? Una función aislada puede “existir”; un flujo permite que una persona alcance un objetivo sin quedar atrapada.

Antes de continuar, señale en su dibujo dónde empieza el objetivo, qué decisión toma la persona y cómo sabe que terminó.

Prompt de IA permitido · Validar requisitos. Actúa como analista de producto. Te describiré un MVP comunitario. No propongas código. Hazme diez preguntas para detectar ambigüedades, separa requisitos funcionales y no funcionales, y señala cualquier función que pertenezca a una fase posterior.

Antes de usar la respuesta: subraye qué decidió usted, qué parte verificará y qué evidencia demostrará que la propuesta funciona.

Control de avance

- ☐ El README contiene el contrato resumido del producto.
- ☐ Existe un flujo dibujado con éxito, error y regreso.
- ☐ Puede explicar por qué autenticación y backend propio no pertenecen a este módulo.

Evidencia mínima

Captura o enlace del flujo y primer apartado del README.

Commit sugerido

```
docs: definir problema, alcance y flujo del MVP
```

Parte 1 Preparar el entorno y crear el proyecto

CONECTA CON LA CLASE 1

El CLI automatiza tareas repetibles; no toma decisiones de arquitectura por usted. Node ejecuta las herramientas de desarrollo, npm administra paquetes y Angular CLI crea, sirve, compila y mantiene el workspace.

Actividad 1.1 Valide antes de instalar

Abra la terminal integrada de VS Code. En Windows puede usar Ctrl + Ñ; en otros teclados, View → Terminal. Ejecute una instrucción a la vez y lea el resultado.

```
node --version
npm --version
ng version
```

Según el entorno trabajado en clase, Angular 22 requiere una versión compatible de Node: 20.19 o superior dentro de la rama 20, 22.12 o superior dentro de la rama 22, o 24. Si ng no existe, instale el CLI globalmente.

```
npm install -g @angular/cli
```

No avance por inercia. Si un comando falla, lea la primera causa útil. “command not found” no es un error de Angular: indica que la herramienta no está instalada o no está disponible en la variable PATH.

Actividad 1.2 Cree el workspace

```
ng new conecta-barrio
```

En las preguntas del asistente seleccione CSS, mantenga componentes standalone y no active renderizado en servidor para este MVP. La integración de herramientas de IA del CLI es opcional; no reemplaza la bitácora solicitada.

```
cd conecta-barrio
code .
ng serve
```

Abra <http://localhost:4200>. Para detener el servidor use Ctrl + C. Si el puerto 4200 está ocupado, puede ejecutar:

```
ng serve --port 4300
```

Actividad 1.3 Cree el primer punto de control Git

```
git status
```

Si el CLI no inició Git en su entorno, ejecute git init. Después guarde el estado inicial.

```
git add .
git commit -m "chore: iniciar proyecto Angular de ConectaBarrio"
```

Atajos que reducen fricción

Atajo	Uso
Ctrl + P	Abrir rápidamente un archivo por nombre.
Ctrl + Shift + P	Abrir la paleta de comandos de VS Code.
Ctrl + ` o Ctrl + Ñ	Mostrar u ocultar la terminal integrada, según teclado.
Ctrl + C	Detener ng serve en la terminal activa.
Ctrl + S	Guardar y activar la recompilación.
F12	Ir a la definición de un símbolo.
Shift + Alt + F	Formatear el archivo actual.

Control de avance

- ☐ ng version muestra Angular CLI 22.
- ☐ La app abre en localhost sin errores rojos de compilación.
- ☐ El nombre del proyecto es conecta-barrio.
- ☐ Existe un commit inicial limpio.

Evidencia mínima

Captura del navegador y salida resumida de ng version; no incluya datos personales del equipo.

Parte 2 Crear la estructura sin convertirla en una receta

CONECTA CON LAS CLASES 1 Y 2

Una página representa una ruta completa; un componente representa una pieza reutilizable. Separarlas evita que el componente raíz se convierta en toda la aplicación.

Actividad 2.1 Genere páginas y componente

Use el CLI para crear el esqueleto. Los comandos no resuelven el contenido: solo crean archivos consistentes.

```
ng generate component pages/inicio
ng generate component pages/solicitudes-listado
ng generate component pages/solicitud-detalle
ng generate component pages/solicitud-nueva
ng generate component pages/lideres-listado
ng generate component components/solicitud-card
ng generate service services/solicitudes
ng generate service services/lideres
```

Actividad 2.2 Cree la carpeta de modelos

Cree `src/app/models` y agregue dos archivos: `solicitud.model.ts` y `lider.model.ts`. En ellos describirá los contratos mostrados en la sección 3. No agregue estado, peticiones ni funciones.

Actividad 2.3 Limpie con intención

Retire el contenido de demostración del componente raíz. Conserve el cascarón que alojará navegación, contenido enrutado y pie de página. Antes de borrar algo, identifique para qué servía. Borrar sin comprender produce errores que luego parecen “misteriosos”.

Pausa de criterio

Pregunta. ¿Por qué no crear un servicio para cada botón o una carpeta para cada etiqueta? Porque la arquitectura organiza responsabilidades significativas, no cada detalle visual. Una estructura demasiado fragmentada también dificulta leer el sistema.

Prompt de IA permitido · Revisión de responsabilidades. Te daré una lista de páginas, componentes, modelos y servicios de una aplicación Angular. No escribas código. Devuélveme una tabla con una responsabilidad por pieza, dependencias permitidas y señales de que cada pieza está asumiendo demasiado.

Antes de usar la respuesta: subraye qué decidió usted, qué parte verificará y qué evidencia demostrará que la propuesta funciona.

Control de avance

- ☐ Las cinco páginas, la tarjeta, los dos servicios y los dos modelos existen.
- ☐ El componente raíz no contiene toda la lógica del producto.
- ☐ Puede explicar la diferencia entre página, componente, servicio y modelo.
- ☐ `ng serve` sigue funcionando después de la limpieza.

Evidencia mínima

Captura del explorador de archivos y una frase de responsabilidad por carpeta.

Commit sugerido

```
| chore: crear arquitectura inicial por responsabilidades
```

Parte 3 Diseñar una identidad visual coherente

CONECTA CON HTML, CSS Y SISTEMA DE DISEÑO

Diseñar no significa decorar al final. Un sistema visual pequeño reduce decisiones repetidas y hace que la interfaz comunique jerarquía. El objetivo no es imitar una marca; es construir una experiencia legible y consistente.

Actividad 3.1 Defina tokens globales

En `src/styles.css` defina, con variables CSS, una paleta propia, colores de texto y superficie, borde, sombra, radios y espaciados frecuentes. También establezca tipografía, fondo general y `box-sizing` global.

No copie los colores de VetCare. ConectaBarrio debe tener una identidad distinta. Asegure contraste suficiente entre texto y fondo.

Actividad 3.2 Dibuje cinco estados visuales

- Tarjeta normal.
- Tarjeta marcada como prioritaria.
- Botón con foco de teclado visible.
- Mensaje de carga o espera.
- Mensaje de error con acción para reintentar.

Actividad 3.3 Piense primero en celular

La grilla debe pasar de una columna a varias cuando haya espacio. Evite anchos fijos que obliguen a desplazarse horizontalmente. Los controles táctiles deben tener área suficiente y el texto no debe depender del hover para comprender una acción.

Recordatorio de CSS. Los estilos globales expresan reglas compartidas. Los estilos de una página o componente deben permanecer junto a esa pieza. Si una clase solo tiene sentido dentro de solicitud-card, no pertenece a styles.css.

Control de avance

- ☐ Los tokens visuales se definen una sola vez.
- ☐ La paleta tiene contraste legible.
- ☐ La página no genera desplazamiento horizontal en ancho móvil.
- ☐ El foco de teclado es visible en enlaces, botones y campos.

Evidencia mínima

Capturas a 390 px y a 1280 px de ancho, aunque el contenido todavía sea provisional.

Commit sugerido

```
|style: definir sistema visual responsive de ConectaBarrio
```

Parte 4 Construir el cascarón y la navegación

CONECTA CON LA CLASE 2

En una SPA, el navegador no solicita una página HTML diferente por cada vista. Angular cambia el componente asociado a la URL dentro del router-outlet. La URL sigue siendo importante: permite volver, compartir y abrir directamente un estado de navegación.

Actividad 4.1 Configure las rutas

En app.routes.ts traduzca el mapa de rutas de la sección 4. Use loadComponent para las páginas obligatorias. El orden importa: una ruta comodín, si la implementa, debe quedar al final.

Actividad 4.2 Configure el binding de parámetros

En app.config.ts conserve el proveedor del router y habilite el binding de parámetros hacia inputs del componente. Más adelante, la página de detalle recibirá id desde la ruta. No configure HttpClient todavía si aún no comprende por qué; se hará en la Parte 10.

Actividad 4.3 Construya el cascarón

En app.html construya una cabecera con nombre del producto, navegación a Inicio, Solicitudes, Nueva solicitud y Líderes, un lugar para el contador de prioridades, el router-outlet y un pie breve. Use routerLink para navegar dentro de la aplicación.

Actividad 4.4 Pruebe como usuaria

- ☐ Cada enlace cambia la URL sin recargar toda la página.
- ☐ El botón Atrás del navegador recorre el historial correctamente.
- ☐ Abrir /solicitudes/1 directamente muestra la página de detalle, aunque aún tenga contenido provisional.
- ☐ Refrescar localhost en una ruta interna no rompe la aplicación.
- ☐ La navegación puede recorrerse solo con Tab y Enter.

Pausa de criterio

routerLink frente a href. Un href tradicional puede provocar una navegación completa y reiniciar el estado en memoria. routerLink pide al Router de Angular cambiar la vista dentro de la SPA. Saber por qué usa una herramienta es más importante que recordar su sintaxis.

Control de avance

- ☐ Las cinco rutas obligatorias son accesibles.
- ☐ Las páginas se cargan de forma diferida.
- ☐ El cascarón contiene un único router-outlet.
- ☐ La navegación no usa clics sobre elementos que no son enlaces o botones.

Evidencia mínima

Grabación breve o secuencia de capturas que muestre URL, navegación y botón Atrás.

Commit sugerido

```
feat: configurar rutas y cascaron navegable
```

Parte 5 Modelar datos antes de mostrarlos

CONECTA TYPESCRIPT CON EL CONTRATO REST

TypeScript no cambia la respuesta de la API. Le ayuda a detectar, durante el desarrollo, si está usando un campo que no existe o con un tipo incorrecto. Una interfaz es un contrato de forma; no es una copia de datos.

Actividad 5.1 Modele la solicitud

En solicitud.model.ts describa los cuatro campos del recurso posts: id, userId, title y body, con los tipos indicados en la sección 3. Si decide crear un tipo separado para el cuerpo de creación, explique por qué todavía no contiene id.

Actividad 5.2 Modele el líder

En lider.model.ts describa únicamente los campos que realmente utilizará. Si usa campos anidados como address.city o company.name, represente su estructura. No convierta todo en any para silenciar el editor.

Actividad 5.3 Trace una matriz de traducción

Campo remoto	Etiqueta visible sugerida	Decisión que debe justificar
title	Asunto	Cuánto texto mostrar en tarjeta antes de truncar.
body	Descripción	Cómo conservar legibilidad en lista y detalle.
userId	Líder relacionado	Cómo resolver el nombre sin duplicar datos.
id	Caso #	Cómo tratar un id inválido o inexistente.

Error común

La propiedad no existe. Si TypeScript informa que una propiedad no existe en el tipo, no agregue el campo a ciegas. Compare primero el JSON real, el modelo y el template. El error suele revelar que el contrato y la interfaz se separaron.

Control de avance

- ☐ No se utiliza any para los recursos principales.
- ☐ Los nombres y tipos coinciden con la API.
- ☐ El tipo de creación no exige un id generado por el cliente.
- ☐ Puede explicar la diferencia entre contrato remoto y lenguaje de la interfaz.

Evidencia mínima

Captura del editor sin errores de tipos y matriz de traducción incluida en README.

Commit sugerido

|feat: modelar contratos de solicitudes y lideres

Parte 6 Diseñar la fuente única de verdad con Signals

CONECTA CON LA CLASE 3

El estado es la información que puede cambiar mientras se usa la aplicación. Si cada página mantiene su propia copia, dos vistas pueden mostrar verdades diferentes. El servicio compartido ofrece un lugar único para leer y modificar el estado.

Actividad 6.1 Diseñe el estado de SolicitudesService

Antes de escribir, haga una tabla con los Signals necesarios. Debe distinguir entre estado escribible privado, vista pública de solo lectura y valores derivados.

Estado	Clase conceptual	Responsabilidad
Colección de solicitudes	Signal escribible privado + vista readonly	Guardar la respuesta actual y protegerla de escrituras externas.
Ids prioritarios	Signal escribible privado	Representar selección local sin alterar el contrato remoto.
Total de prioridades	computed	Derivar un número; no mantener una segunda variable sincronizada a mano.
Cargando	Signal booleano	Indicar que una petición está en curso.
Mensaje de error	Signal de texto o ausencia	Explicar fallos recuperables.
Última creación	Estado opcional	Dar retroalimentación sin confundirla con persistencia permanente.

Actividad 6.2 Defina operaciones del servicio

Nombre operaciones por intención: cargarSolicitudes, crearSolicitud, alternarPrioridad, esPrioritaria y obtenerPorId son ejemplos de lenguaje del dominio. Usted puede ajustar nombres, pero evite métodos genéricos como hacerCosa o procesar.

Actividad 6.3 Actualice de forma inmutable

Al cambiar un Set o un arreglo dentro de una Signal, cree una referencia nueva. Mutar el mismo objeto puede impedir que Angular notifique el cambio. Piense el proceso como: leer → copiar → modificar la copia → actualizar.

Pausa de criterio

Pregunta. ¿Por qué el total de prioridades debe ser computed y no otro Signal que se incremente y decremente?
Orientación conceptual: si un valor puede calcularse completamente desde otro estado, guardarlo por separado crea dos fuentes de verdad que pueden desincronizarse.

Prompt de IA permitido · Auditar estado. Sin escribir código, revisa este inventario de estado para una app Angular con Signals. Señala qué valores deben ser fuente de verdad, cuáles derivados con computed, qué datos deben exponerse como readonly y dónde existe riesgo de mutación de arreglos o Set.

Antes de usar la respuesta: subraye qué decidió usted, qué parte verificará y qué evidencia demostrará que la propuesta funciona.

Control de avance

- ☐ El servicio es providedIn root y las páginas lo obtienen mediante inject.
- ☐ La colección escribible no queda expuesta públicamente.
- ☐ El total de prioridades es derivado.
- ☐ Las actualizaciones crean nuevas referencias.
- ☐ No hay una segunda colección de solicitudes en otra página.

Evidencia mínima

Diagrama actualizado del flujo: servicio → página → tarjeta → evento → servicio.

Commit sugerido

|feat: centralizar estado de solicitudes con Signals

Parte 7 Construir listado, búsqueda y tarjeta reutilizable

INTEGRA CLASES 1 Y 3

La página de listado coordina la colección y el buscador. La tarjeta presenta una unidad y comunica una intención. El componente hijo no debe adivinar de dónde vienen los datos ni editar directamente el estado del padre.

Actividad 7.1 Diseñe SolicitudCard

La tarjeta debe recibir una solicitud y un indicador de prioridad. Debe emitir un evento cuando la persona solicite cambiar esa prioridad. Muestre id, asunto, una vista breve de la descripción y una acción clara para abrir el detalle. El botón de prioridad debe informar su estado a tecnologías de asistencia con aria-pressed o una etiqueta equivalente. Si tarjeta y botón son elementos interactivos anidados, controle el event bubbling o, preferiblemente, separe las zonas de interacción.

Actividad 7.2 Diseñe el estado local de búsqueda

El texto de búsqueda pertenece al listado, no al servicio global. Representélo con una Signal local. Derive solicitudesFiltradas con computed a partir del texto y la colección pública del servicio. Compare sin distinguir mayúsculas/minúsculas y elimine espacios accidentales.

Actividad 7.3 Represente los estados

Condición	Mensaje/experiencia	Error que evita
Cargando	Indicador y texto que explica que se consultan solicitudes.	Mostrar “sin resultados” antes de recibir la API.
Error	Mensaje útil y acción de reintento.	Dejar la pantalla vacía.
Colección vacía	Explicación y enlace a registrar.	Confundir ausencia de datos con fallo.

Condición	Mensaje/experiencia	Error que evita
Búsqueda sin coincidencias	Conservar el buscador y ofrecer limpiar filtro.	Hacer creer que la base completa está vacía.
Datos disponibles	Grilla de tarjetas y cantidad visible.	Mostrar información sin jerarquía.

Actividad 7.4 Compruebe Input y Output

Cambie prioridad en una tarjeta. El contador global y la apariencia de esa misma tarjeta deben actualizarse sin recargar. Después abra otra ruta y regrese: mientras no recargue el navegador, el estado compartido debe mantenerse.

Pausa de criterio

Datos bajan, eventos suben. La página pasa la solicitud y el estado de prioridad hacia la tarjeta. La tarjeta no modifica al padre: emite que ocurrió una intención. La página o el servicio decide cómo cambiar el estado. Este flujo hace más predecible la aplicación.

Control de avance

- ☐ La tarjeta se reutiliza dentro de @for.
- ☐ La tarjeta recibe datos y emite la acción de prioridad.
- ☐ El buscador usa signal y el resultado usa computed.
- ☐ Los cinco estados de lista se distinguen.
- ☐ El botón de prioridad funciona con teclado y comunica su estado.

Evidencia mínima

Video corto: buscar, limpiar, marcar prioridad y observar el contador.

Commit sugerido

```
feat: crear listado reactivo y tarjeta reutilizable
```

Parte 8 Construir el detalle mediante parámetro de ruta

CONECTA ROUTING CON ESTADO COMPARTIDO

Una ruta como /solicitudes/12 expresa una identidad. El parámetro llega como texto, aunque represente un número. La página debe validar esa entrada y diferenciar “todavía cargando” de “el id no existe”.

Actividad 8.1 Reciba y valide el id

Use el binding de parámetros habilitado en app.config.ts. Convierta el id conscientemente y considere qué ocurre con valores vacíos, no numéricos, cero o fuera del rango disponible.

Actividad 8.2 Derive la solicitud

Obtenga la solicitud desde la colección compartida. Si el servicio aún está cargando, espere. Cuando la carga terminó y no existe coincidencia, muestre un estado “caso no encontrado” con enlace al listado. No haga que cada visita dispare peticiones duplicadas sin necesidad.

Actividad 8.3 Complete el flujo

El detalle debe mostrar asunto, descripción completa, id, líder relacionado si está disponible, prioridad actual, acción para alternarla y una forma clara de regresar. El cambio debe verse también al volver al listado.

Pregunta de criterio

¿Por qué no basta con buscar inmediatamente y mostrar “no existe”? Porque al iniciar la aplicación la colección puede estar temporalmente vacía mientras viaja la petición. Concluir demasiado pronto mezcla un estado transitorio con un resultado definitivo.

Control de avance

- ☐ /solicitudes/1 abre un detalle real.
- ☐ Un id inválido no rompe la consola ni deja pantalla blanca.
- ☐ Carga y no encontrado son estados diferentes.
- ☐ Cambiar prioridad en detalle actualiza el contador y el listado.

Evidencia mínima

Capturas de un detalle válido y un id inexistente, con URL visible.

Commit sugerido

```
|feat: implementar detalle por parametro de ruta
```

Parte 9 Convertir Inicio en un tablero útil

REUTILIZA ESTADO SIN DUPLICARLO

La página de inicio no debe ser un cartel decorativo. Debe orientar, resumir y ofrecer el siguiente paso. Sus indicadores leen el mismo estado compartido que el listado.

Contenido mínimo

- Una frase que explique el problema que resuelve ConectaBarrio.
- Acciones para ver solicitudes y registrar una nueva.
- Total de solicitudes cargadas.
- Total de solicitudes prioritarias.
- Estado de conexión o mensaje prudente mientras carga.
- Una nota visible sobre el carácter demostrativo del MVP.

Decisión técnica

No mantenga totales manuales dentro de Inicio. Lea Signals o valores computed del servicio. Si mañana cambia la colección, el tablero debe responder automáticamente.

Control de avance

- ☐ Inicio ofrece contexto, resumen y siguiente acción.

- ☐ Los indicadores cambian al marcar prioridad.
- ☐ No hay cálculos duplicados entre Inicio y el cascarón.
- ☐ La interfaz sigue siendo legible sin depender de imágenes externas.

Evidencia mínima

Captura de Inicio antes y después de marcar una prioridad.

Commit sugerido

```
feat: crear tablero de inicio conectado al estado
```

Parte 10 Conectar HttpClient y cargar datos reales

CONECTA CON LA CLASE 4

Hasta aquí pudo trabajar con una colección temporal mínima para diseñar la interfaz. Ahora la fuente cambia: el dato vive en otro proceso, tarda y puede fallar. La arquitectura debe permitir cambiar el origen sin reescribir las páginas.

Actividad 10.1 Habilite HttpClient

Registre provideHttpClient una sola vez en app.config.ts. Si Angular indica que no existe un proveedor para HttpClient, revise este archivo antes de modificar los servicios.

Actividad 10.2 Implemente el GET en SolicitudesService

Centralice la URL base o la URL del recurso en el servicio. Al iniciar la carga: limpie el error anterior y active cargando. En la respuesta exitosa: actualice la colección. En error: guarde un mensaje comprensible. En todos los casos: asegure que cargando vuelva a falso.

Actividad 10.3 Implemente el GET en LideresService

Repita el patrón con /users sin copiar a ciegas. Compare ambos servicios: misma estructura asincrónica, contratos y responsabilidades diferentes. La página de líderes debe leer el estado público y mostrar nombre, correo, teléfono, ciudad y organización disponibles.

Observable frente a Signal

Concepto	Pregunta que responde	Uso en este taller
Observable de HttpClient	¿Qué llegará en el futuro desde la red?	Recibir respuesta o error de una petición.
Signal	¿Cuál es el valor actual que la interfaz debe observar?	Mantener solicitudes, líderes y estado de UI.
subscribe	¿Qué hacer cuando llega éxito o error?	Puente controlado entre la respuesta remota y los Signals.
computed	¿Qué valor puede derivarse del estado actual?	Filtros, totales y selección por id.

Actividad 10.4 Pruebe la red

- Abra las herramientas del navegador con F12 y la pestaña Network.
- Dev Senior Code · © 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · [linkedin.com/in/ana-alvarado-instructora-full-stack](https://www.linkedin.com/in/ana-alvarado-instructora-full-stack) · 22 / 39

- 2. Recargue /solicitudes y localice la petición a /posts.
- 3. Compruebe método, estado HTTP, tiempo y respuesta.
- 4. Desactive temporalmente la red o altere la URL de desarrollo para provocar un fallo.
- 5. Verifique que la interfaz explica el error y permite reintentar.
- 6. Restaure la URL correcta y confirme recuperación.

Prompt de IA permitido · Interpretar un error HTTP. Tengo una aplicación Angular con HttpClient. Te compartiré el mensaje exacto, el estado HTTP y lo que muestra Network. No escribas una solución completa. Formula hasta cinco hipótesis ordenadas, qué evidencia confirmaría cada una y qué archivo debería revisar primero.
Antes de usar la respuesta: subraye qué decidió usted, qué parte verificará y qué evidencia demostrará que la propuesta funciona.

Control de avance

- ☐ HttpClient se provee una sola vez.
- ☐ Las peticiones viven en servicios, no en páginas.
- ☐ GET /posts y GET /users aparecen en Network.
- ☐ Carga, éxito y error actualizan Signals.
- ☐ La app puede recuperarse después de un fallo simulado.

Evidencia mínima

Capturas de Network para una petición exitosa y de la interfaz en error recuperable.

Commit sugerido

```
feat: consumir solicitudes y lideres desde API REST
```

Parte 11 Registrar una solicitud con POST

COMPLETA EL CICLO DE LECTURA Y ESCRITURA

El formulario no existe para demostrar campos HTML. Existe para completar un caso de negocio: registrar una necesidad y reflejarla inmediatamente en el trabajo de la coordinadora.

Actividad 11.1 Diseñe el formulario

Campo visible	Campo del contrato	Validación mínima
Líder	userId	Selección obligatoria de un líder cargado.
Asunto	title	Obligatorio, sin solo espacios y con longitud razonable.
Descripción	body	Obligatoria, con contexto suficiente.

Puede usar referencias de plantilla, SubmitEvent y preventDefault, siguiendo lo trabajado en clase. No introduzca una biblioteca de formularios no estudiada solo para reducir unas líneas.

Actividad 11.2 Valide en dos niveles

El navegador puede apoyar con required y tipos adecuados. La lógica de la página debe impedir enviar valores vacíos o un líder inexistente. En un sistema real, el backend también validaría: nunca se confía solo en el frontend.

Actividad 11.3 Envíe y actualice

La página entrega un objeto de creación al servicio. El servicio realiza POST /posts. Cuando la API responde, agregue el objeto devuelto a la colección de forma inmutable. Muestre confirmación, limpie o restablezca el formulario y ofrezca abrir el listado.

Actividad 11.4 Comunique la simulación

Informe que el caso se agregó a la sesión actual y que la API de práctica no garantiza persistencia después de recargar. No use la palabra “guardado permanentemente”.

Prueba del flujo

- ☐ Enviar vacío no genera una petición.
- ☐ Una selección de líder válida se traduce al userId correcto.
- ☐ Network muestra POST /posts y un cuerpo coherente.
- ☐ La respuesta aparece en la colección sin recargar.
- ☐ El estado de carga del botón evita envíos repetidos accidentales.
- ☐ Un fallo de POST conserva los datos necesarios para volver a intentar.
- ☐ Después de recargar, la interfaz explica por qué el elemento simulado puede desaparecer.

Pausa de criterio

¿Quién crea el id?. En una operación de creación, el cliente envía los datos necesarios y el servidor asigna la identidad. Inventar el id en el formulario mezcla responsabilidades y puede producir colisiones cuando exista una base real.

Control de avance

- ☐ El formulario es accesible por etiqueta y teclado.
- ☐ La validación ocurre antes de llamar al servicio.
- ☐ El servicio realiza el POST y actualiza la colección inmutablemente.
- ☐ La interfaz diferencia éxito, envío en curso y error.
- ☐ La limitación de persistencia está documentada.

Evidencia mínima

Video corto desde formulario hasta aparición del caso en el listado, con Network abierto.

Commit sugerido

```
|feat: registrar solicitudes mediante POST simulado
```

Parte 12 Persistir prioridades como reto de consolidación

EXTENSIÓN RECOMENDADA · BASADA EN EFFECT

En clase, effect se presentó como una herramienta para sincronizar Signals con algo externo, como localStorage. Este reto no sustituye ninguna función obligatoria; fortalece el manejo de estado.

Comportamiento esperado

- Al marcar o desmarcar prioridad, la colección de ids se guarda en localStorage.
- Al iniciar el servicio, se recuperan únicamente ids válidos.
- Recargar conserva prioridades, aunque el POST simulado no persista.
- Si el contenido guardado está corrupto, la app usa un estado seguro sin romperse.

Criterio técnico

computed devuelve un valor derivado. effect ejecuta un efecto secundario y no debe usarse para calcular valores. Ubíquelo en un contexto de inyección válido y evite que escriba sobre las mismas Signals que lee sin una razón controlada.

Control de avance

- ☐ Recargar conserva prioridades.
- ☐ Borrar la clave o guardar un valor inválido no rompe la app.
- ☐ El servicio sigue siendo la única fuente de verdad.

Evidencia mínima

Captura de Application → Local Storage y demostración antes/después de recargar.

Commit sugerido

```
feat: persistir prioridades localmente con effect
```

Parte 13 Calidad, accesibilidad y pruebas manuales

PENSAR COMO QUIEN ENTREGA SOFTWARE

Que una función opere en el caso ideal no significa que el producto esté listo. La calidad aparece cuando el sistema también explica la espera, tolera entradas extrañas, permite navegar con teclado y conserva su estructura en diferentes tamaños.

Lista de accesibilidad

- ☐ Existe un solo h1 significativo por página y los encabezados siguen una jerarquía.
- ☐ Todos los campos tienen etiqueta visible; el placeholder no actúa como única etiqueta.
- ☐ Botones y enlaces se distinguen por su propósito, no solo por color.
- ☐ El foco de teclado es visible y el orden de Tab es lógico.
- ☐ El botón de prioridad comunica estado con aria-pressed o alternativa equivalente.
- ☐ Los mensajes de error explican qué ocurrió y qué puede hacer la persona.
- ☐ El contraste permite leer texto normal y controles.
- ☐ La interfaz funciona al 200 % de zoom sin perder acciones esenciales.

Matriz de pruebas obligatoria

ID	Escenario	Acción	Resultado esperado
P-01	Inicio limpio	Abrir la URL base.	Se ve propósito, navegación y estado inicial comprensible.
P-02	Carga de solicitudes	Abrir /solicitudes con red normal.	Se muestra espera y luego tarjetas.
P-03	Búsqueda con resultado	Ingresa una palabra existente.	La lista se reduce sin petición adicional.
P-04	Búsqueda sin resultado	Ingresa una palabra inexistente.	Mensaje específico y opción de limpiar.
P-05	Prioridad	Alternar desde una tarjeta.	Tarjeta, contador e Inicio se sincronizan.
P-06	Detalle válido	Abrir una tarjeta.	URL incluye id y muestra el caso correcto.
P-07	Detalle inválido	Abrir /solicitudes/9999.	No hay pantalla blanca; existe regreso.
P-08	GET fallido	Simular red offline y recargar.	Error comprensible y reintento.
P-09	Formulario vacío	Enviar sin completar.	No sale POST; se indican campos.
P-10	POST válido	Completar y enviar.	Network muestra POST y el caso aparece en sesión.
P-11	Teclado	Recorrer flujo sin mouse.	Todas las acciones son alcanzables y visibles.
P-12	Móvil	Probar a 390 px.	Sin scroll horizontal ni controles cortados.
P-13	Ruta profunda	Refrescar /solicitudes/1 desplegada.	El hosting entrega la SPA y el detalle abre.

Registro de hallazgos

Por cada prueba que falle, registre: resultado real, resultado esperado, hipótesis, cambio realizado y nueva evidencia. Un bug documentado y corregido demuestra más criterio que una captura aislada de “funciona”.

Prompt de IA permitido · Pruebas adversas. Actúa como QA de una SPA Angular. A partir de estas historias de usuario, propón casos de éxito, borde y fallo. No escribas código ni pruebas automatizadas. Prioriza rutas, red lenta, error HTTP, entrada inválida, teclado, móvil y recarga de URL profunda. Antes de usar la respuesta: subraye qué decidió usted, qué parte verificará y qué evidencia demostrará que la propuesta funciona.

Control de avance

- ☐ Las 13 pruebas tienen resultado registrado.
- ☐ Los fallos encontrados se cerraron o quedaron documentados como limitación.
- ☐ El flujo principal puede completarse solo con teclado.
- ☐ No hay errores inesperados en consola durante el recorrido normal.

Evidencia mínima

Matriz completada y al menos un antes/después de un defecto corregido.

Commit sugerido

test: validar flujos, errores y accesibilidad

Parte 14 Documentar y preparar el repositorio

UNA ENTREGA PROFESIONAL TAMBIÉN SE PUEDE COMPRENDER

El README no es una tarea decorativa. Es la puerta de entrada para quien evalúa, mantiene o continúa el producto. Debe explicar cómo ejecutar, qué decisiones se tomaron y qué limitaciones existen.

Contenido obligatorio del README

- ☐ Nombre y propósito del producto.
- ☐ Problema, usuario principal y alcance.
- ☐ Captura principal.
- ☐ Tecnologías utilizadas.
- ☐ Requisitos de entorno y comandos para instalar/ejecutar.
- ☐ Mapa de rutas.
- ☐ Arquitectura de carpetas y responsabilidades.
- ☐ API utilizada, endpoints y limitación del POST simulado.
- ☐ Decisiones sobre Signals, estado compartido e Input/Output.
- ☐ Pruebas manuales ejecutadas.
- ☐ URL de despliegue.
- ☐ Limitaciones conocidas y siguientes pasos.
- ☐ Sección “Uso de IA” con enlace o resumen de la bitácora.

Historial de commits

El historial debe contar la evolución del producto. Evite un único commit llamado “final”. No necesita decenas de commits; necesita puntos de recuperación con intención clara.

Prefijo sugerido	Cuándo usarlo	Ejemplo de intención
chore	Configuración o estructura.	crear arquitectura inicial
feat	Comportamiento nuevo para la persona usuaria.	agregar detalle por ruta
fix	Corrección de un defecto.	distinguir carga de lista vacía
style	Cambios visuales sin nueva lógica.	ajustar grilla responsive
docs	Documentación.	explicar API simulada
test	Evidencia o matriz de pruebas.	registrar validación manual

Revisión antes de publicar

```
git status
git log --oneline --decorate -10
```

Compruebe que no subió archivos de entorno con secretos, carpetas generadas innecesarias ni datos personales. Este proyecto no necesita claves para JSONPlaceholder.

Control de avance

- ☐ README permite ejecutar el proyecto sin explicación oral.
- ☐ El historial contiene hitos comprensibles.
- ☐ git status está limpio antes del despliegue.
- ☐ La bitácora de IA distingue ayuda, decisión y verificación.

Evidencia mínima

URL del repositorio accesible para la instructora y captura del historial.

Commit sugerido

```
docs: completar guía de uso y decisiones del proyecto
```

Parte 15 Compilar y desplegar

DE LOCALHOST A UNA URL REVISABLE

ng serve es un servidor de desarrollo. Para entregar, Angular produce una compilación optimizada. El hosting debe servir los archivos generados y reenviar rutas desconocidas a index.html para que el Router resuelva las URLs profundas.

Actividad 15.1 Compile producción

```
ng build
```

No ignore advertencias de presupuesto o errores de TypeScript. Revise la carpeta de salida indicada por la terminal y por angular.json. En proyectos recientes suele ubicarse bajo dist/conecta-barrio/browser, pero la configuración del proyecto es la fuente de verdad.

Actividad 15.2 Publique el repositorio

Cree un repositorio remoto y conecte el proyecto local. Sustituya la URL del ejemplo por la suya.

```
git branch -M main
git remote add origin URL_DEL_REPOSITORIO
git push -u origin main
```

Actividad 15.3 Importe en Vercel

1. Inicie sesión en Vercel y seleccione Add New → Project.
2. Importe el repositorio de ConectaBarrio desde GitHub.
3. Compruebe que el framework detectado sea Angular.
4. Use el comando de build del proyecto. Si la detección automática falla, confirme ng build.
5. Confirme el directorio de salida real. No lo adivine: léalo en la compilación o en angular.json.
6. Despliegue y espere el estado Ready.

Actividad 15.4 Pruebe el sitio como evaluador

- ☐ La URL raíz abre por HTTPS.
- ☐ GET /posts y GET /users funcionan desde el dominio desplegado.
- ☐ El POST devuelve respuesta y la interfaz comunica que es una simulación.
- ☐ Abrir y refrescar /solicitudes/1 no devuelve 404 del hosting.
- ☐ Los estilos y assets cargan.
- ☐ La consola no presenta errores de producción durante el flujo normal.
- ☐ La URL desplegada está en el README.

Si una ruta profunda muestra 404. La navegación interna puede funcionar y, aun así, una recarga fallar porque el servidor busca un archivo físico con ese nombre. El hosting debe usar un fallback hacia index.html. Consulte la configuración oficial de la plataforma y vuelva a probar escribiendo la URL directamente.

Control de avance

- ☐ ng build finaliza sin errores.
- ☐ El repositorio remoto refleja el commit evaluado.
- ☐ La URL pública completa el flujo principal.
- ☐ Una ruta parametrizada sobrevive a la recarga.

Evidencia mínima

URL pública, URL del repositorio, fecha/hora del último deploy y captura del estado Ready.

Commit sugerido

```
|chore: preparar version desplegable del MVP
```

Parte 16 Defensa técnica y retrospectiva

LA EVIDENCIA FINAL ES COMPRENDER LO CONSTRUIDO

La defensa no busca recitar sintaxis. Busca comprobar que puede recorrer el sistema con un modelo mental: qué inicia una acción, dónde cambia el estado, qué pieza reacciona y cómo se verifica el resultado.

Recorrido de demostración sugerido 7 minutos

1. Explique el problema de RedManos y el alcance del MVP.
2. Muestre Inicio y los indicadores.
3. Abra Solicitudes, busque y marque una prioridad.
4. Abra el detalle por URL y regrese.
5. Registre una solicitud y explique la simulación del POST.
6. Provoque o muestre evidencia de un error de red manejado.
7. Señale arquitectura, historial Git y URL desplegada.

Preguntas posibles de defensa

- ¿Por qué el buscador es estado local y las prioridades son estado compartido?
- ¿Qué problema resuelve asReadonly?
- ¿Por qué el total de prioridades es computed?
- ¿Qué sucede desde que se envía el evento de la tarjeta hasta que cambia el contador?
- ¿Por qué una petición HTTP no entrega inmediatamente el dato?
- ¿Qué diferencia hay entre el Observable de HttpClient y el Signal visible en la interfaz?
- ¿Cómo distingue la página Detalle entre cargando y no encontrado?
- ¿Por qué no se realiza el GET directamente en el componente?
- ¿Qué garantiza y qué no garantiza el POST de JSONPlaceholder?
- ¿Qué cambiaría para conectar un backend Spring Boot real?
- ¿Por qué routerLink es preferible a href en esta navegación interna?
- ¿Qué riesgo existe al mutar el mismo Set dentro de una Signal?
- ¿Qué comprobaría si la URL desplegada funciona, pero refrescar una ruta interna falla?
- ¿Qué propuesta de IA rechazó y con qué evidencia?

Retrospectiva escrita

Agregue al README o a un archivo de evidencias respuestas breves y concretas:

- ☐ La decisión técnica que más ayudó a evitar duplicación.
- ☐ El error que más le enseñó y cómo lo diagnosticó.
- ☐ Una parte que inicialmente no comprendía y ahora puede explicar.
- ☐ Una función que decidió no construir para proteger el alcance.
- ☐ La siguiente mejora que haría con una semana adicional.

6. Entregables

Entregable	Contenido	Formato
Repositorio	Código fuente, historial, README y archivos necesarios para ejecutar.	URL accesible para la instructora.
Aplicación desplegada	Versión evaluada con rutas y consumo de API funcional.	URL HTTPS.
README	Problema, instalación, arquitectura, rutas, API, pruebas, decisiones, límites y URL.	Markdown dentro del repositorio.
Evidencias	Matriz de pruebas, capturas clave y un defecto corregido.	Carpeta docs/evidencias o sección enlazada.
Bitácora de IA	Tres o más interacciones con validación y aprendizaje.	Markdown o PDF enlazado desde README.
Defensa	Demostración y respuestas sobre decisiones.	Presentación en vivo o medio definido por la instructora.

Convención de entrega sugerida

Nombre del repositorio: conecta-barrio-nombre-apellido. Nombre del archivo de bitácora: docs/bitacora-ia.md. No incluya números de documento, credenciales ni datos sensibles en nombres, capturas o commits.

7. Rúbrica de evaluación 100 puntos

La rúbrica es visible desde el inicio para que pueda orientar decisiones. Una función no obtiene el puntaje completo solo por existir: debe estar ubicada, comprobada, documentada y ser explicable.

Criterio	Puntos	Evidencia de logro completo
Comprensión del problema y alcance	6	Historias y flujo coherentes; distingue MVP de futuras fases y documenta límites.
Entorno, estructura y arquitectura	10	Proyecto Angular 22 ejecutable; separación clara de páginas, componentes, modelos y servicios.
Componentes y data binding	10	Tarjeta reutilizable, control flow moderno, bindings correctos y sin duplicación de markup.
Routing y navegación	14	Rutas obligatorias, parámetro id, binding, lazy loading, regreso seguro y ruta profunda funcional.
Signals y estado compartido	17	Fuente única de verdad, readonly, computed, inmutabilidad, estado local correcto y contador sincronizado.
Comunicación entre componentes	8	Datos bajan por Input; intención sube por output; event bubbling y accesibilidad controlados.
Consumo de API GET/POST	18	HttpClient en servicios, contratos tipados, GET de dos recursos, POST, actualización local y limitación explicada.
Estados asíncronicos y manejo de errores	7	Distingue carga, vacío, búsqueda sin resultados, no encontrado y error; ofrece recuperación.
Experiencia visual y accesibilidad	5	Diseño coherente, responsive, foco visible, semántica y formularios comprensibles.
Calidad de entrega	5	Build, deploy, README, Git, pruebas y bitácora permiten reproducir y revisar el trabajo.

Niveles de desempeño

Nivel	Rango	Descripción
Sobresaliente	90–100	El flujo es completo y estable; las decisiones están justificadas; la evidencia permite confiar en el resultado.
Competente	80–89	Cumple el flujo principal y la arquitectura; existen detalles menores sin impacto crítico.
En desarrollo	70–79	Hay comprensión parcial y un MVP recorrible, pero faltan criterios importantes o estabilidad.
Requiere refuerzo	Menos de 70	Los vacíos impiden completar el flujo o explicar la relación entre las piezas.

Condiciones críticas

Para considerar el taller entregado. Debe existir repositorio accesible, aplicación que compile, URL desplegada o evidencia verificable de un bloqueo externo, navegación básica, GET de solicitudes y defensa de autoría. La ausencia de una condición crítica limita el nivel global aunque existan elementos visuales.

Distinciones opcionales

Las mejoras siguientes permiten demostrar profundidad, pero no compensan una historia obligatoria rota: persistencia segura de prioridades con effect, página 404, reintento explícito, skeleton de carga accesible, filtro por líder usando estado derivado, indicador de conexión y una medición razonada de rendimiento.

8. Lista final de entrega

- ☐ Puedo explicar el problema sin mencionar primero la tecnología.
- ☐ Todas las historias obligatorias tienen una evidencia.
- ☐ ng serve inicia sin errores.
- ☐ ng build finaliza sin errores.
- ☐ No hay código de demostración del CLI que confunda el producto.
- ☐ No hay arreglos duplicados de solicitudes en las páginas.
- ☐ No hay peticiones HTTP en componentes visuales.
- ☐ La tarjeta usa Input y output con flujo comprensible.
- ☐ La lista distingue carga, error, vacío y búsqueda sin coincidencias.
- ☐ El detalle maneja id inválido y dato aún no cargado.
- ☐ El formulario valida y el POST aparece en Network.
- ☐ La interfaz declara que el POST es simulado y temporal.
- ☐ La navegación funciona con teclado.
- ☐ El diseño funciona en móvil y escritorio.
- ☐ La ruta profunda abre después de desplegar y refrescar.
- ☐ README, evidencias y bitácora están completas.
- ☐ El repositorio no contiene secretos ni datos personales.
- ☐ La URL del deploy corresponde al último commit entregado.
- ☐ Puedo defender una decisión que tomé y una propuesta de IA que rechacé.

Anexo A Biblioteca de prompts de apoyo

Adapte estos prompts con evidencia de su proyecto. Ninguno pide una solución completa. Una buena conversación con IA comienza con contexto, restricción y forma de validación.

A.1 Explicar un concepto

Prompt. Explícame [concepto] en tres niveles: definición técnica, explicación sencilla y aplicación dentro de ConectaBarrio. Después hazme tres preguntas de criterio. No escribas código.

A.2 Revisar una arquitectura

Prompt. Te describiré la responsabilidad de mis páginas, componentes y servicios. Identifica duplicación, acoplamiento y estado en el lugar equivocado. No reescribas archivos; responde con riesgos y preguntas.

A.3 Diagnosticar sin adivinar

Prompt. Este es el error exacto, la acción que lo provoca y lo que muestra Network. Formula hipótesis ordenadas. Para cada una, indica qué observación la confirma o descarta y qué archivo revisar.

A.4 Revisar Signals

Prompt. Evalúa mi flujo de estado. Indica qué debe ser signal, computed, estado local o estado compartido; señala mutaciones peligrosas. No generes una implementación.

A.5 Auditar un formulario

Prompt. Revisa estos campos y reglas de negocio. Propón validaciones de frontend, mensajes comprensibles y casos borde. No escribas HTML ni TypeScript.

A.6 Evaluar accesibilidad

Prompt. A partir de esta descripción o captura, crea una lista priorizada de problemas de teclado, foco, semántica, contraste, etiquetas y mensajes. Explica cómo comprobar cada mejora.

A.7 Preparar defensa

Prompt. Actúa como evaluador del Módulo 1 de Angular. Hazme preguntas sobre routing, Signals, Input/Output, HttpClient, GET/POST, errores y despliegue. Espera mi respuesta y luego señala vacíos conceptuales.

A.8 Revisar README

Prompt. Comprueba si este README permite ejecutar y evaluar el proyecto sin hablar conmigo. Señala información faltante, ambigua o que prometa más de lo que el MVP hace. No inventes datos.

Anexo B Guía de diagnóstico

Síntoma	Cómo se interpreta	Primeras comprobaciones
ng no se reconoce	El CLI no está instalado o PATH no lo encuentra.	ng version; instalación global; reiniciar terminal.
Puerto 4200 ocupado	Otro proceso usa el puerto.	Detener proceso anterior o usar --port 4300.
Pantalla blanca	Puede existir error de compilación o ejecución.	Terminal, consola del navegador, ruta actual y componente cargado.
No provider for HttpClient	HttpClient no fue registrado globalmente.	app.config.ts y provideHttpClient.
routerLink no navega	La directiva o ruta no está disponible.	Imports del standalone, ruta escrita y app.routes.ts.
id produce NaN	El parámetro textual no se convirtió o validó.	Valor recibido, conversión y ruta.
Signal no refresca	Se mutó la misma referencia o no se lee como Signal.	Copias inmutables y paréntesis de lectura.
Prioridad activa también navega	El evento del botón sube al contenedor interactivo.	Estructura semántica y propagación del evento.
GET estado 0	La respuesta no llegó al navegador.	Red, URL, CORS, protocolo HTTPS y consola.
404 de la API	Ruta o id remoto no existe.	URL base, endpoint y parámetro.
POST desaparece al recargar	La API simula la creación.	Documentación de JSONPlaceholder y mensaje de interfaz.
Se ve “sin datos” al iniciar	La UI evalúa vacío antes de terminar la carga.	Orden de condiciones y Signal cargando.
Build falla aunque serve funciona	Producción aplica comprobaciones u optimización adicionales.	Primer error del build, tipos y presupuestos.
Ruta desplegada da 404 al refrescar	El hosting no aplica fallback de SPA.	Configuración de rewrites/index.html de la plataforma.

Método breve de depuración

1. Reproduzca el problema de forma consistente.
2. Reduzca el recorrido hasta encontrar la primera diferencia entre esperado y real.
3. Lea terminal, consola y Network; no cambie varios archivos a la vez.
4. Formule una hipótesis comprobable.
5. Cambie una sola causa y repita la prueba.
6. Registre qué aprendió y elimine trazas temporales innecesarias.

Anexo C Glosario del Módulo 1

Término	Definición aplicada
SPA	Aplicación que cambia vistas en el cliente sin solicitar un documento HTML completo por cada navegación.
Componente standalone	Pieza Angular que declara directamente las dependencias de su template, sin NgModule de la aplicación.
Página	Componente asociado a una ruta y responsable de coordinar un caso de uso visible.
Data binding	Conexión entre estado de TypeScript y vista HTML, o entre eventos de la vista y lógica.
Input	Canal por el que un componente padre entrega datos a un hijo.
Output	Canal por el que un hijo comunica que ocurrió una intención o evento.
Signal	Contenedor reactivo con un valor actual que notifica cambios a quienes lo leen.
computed	Signal derivada de otras Signals; se recalcula cuando cambian sus dependencias.
effect	Efecto secundario que se repite cuando cambian las Signals leídas en su interior.
Servicio	Clase inyectable que concentra una responsabilidad compartida, como estado o comunicación HTTP.
Fuente única de verdad	Lugar autoritativo desde el que se lee y modifica un dato compartido.
Inmutabilidad	Actualizar creando una nueva referencia en vez de modificar directamente el objeto actual.
Router	Sistema que asocia URLs con vistas dentro de la aplicación.
Lazy loading	Carga de una página cuando se visita, en vez de incluirla toda en el paquete inicial.
Parámetro de ruta	Segmento variable de la URL que identifica el recurso o contexto de una vista.
HttpClient	Servicio Angular para realizar peticiones HTTP.
Observable	Representación de valores que pueden llegar en el tiempo, usada por HttpClient.
Contrato JSON	Nombres, tipos y estructura que cliente y servidor acuerdan intercambiar.
Estado HTTP	Código que resume el resultado de una petición: éxito, error del cliente o error del servidor.
CORS	Política del navegador que controla si un origen puede leer respuestas de otro.
Build	Proceso que compila y optimiza la aplicación para entrega.
Deep link	URL que abre directamente una vista interna, por ejemplo /solicitudes/12.
MVP	Versión mínima que resuelve un flujo valioso de principio a fin y permite validar decisiones.

Anexo D Plantilla de evidencia

Use esta estructura para cada hito o defecto importante. Mantenga respuestas cortas y verificables.

Campo	Qué registrar
Hito o prueba	Nombre e identificador.
Objetivo	Qué debía poder hacer la persona usuaria.
Estado inicial	Ruta, datos y condiciones de red.
Acción	Pasos exactos ejecutados.
Resultado esperado	Comportamiento definido antes de probar.
Resultado real	Lo observado, sin interpretaciones.
Evidencia	Captura, video, Network, consola o commit.
Decisión	Qué cambió y por qué.
Reprueba	Resultado después del cambio.
Aprendizaje	Qué modelo mental se fortaleció.

Fuentes y material de referencia

Este taller se diseñó a partir del brochure Java Full Stack Developer de Dev Senior Code y las cuatro guías del Módulo 1 suministradas por la instructora: introducción a Angular; routing y navegación; gestión de estado/comunicación; consumo de APIs y conexión con backend.

[Angular · CLI](#)

[Angular · Deployment](#)

[Angular · Build](#)

[JSONPlaceholder · Guide](#)

[Vercel · Angular](#)

Las plataformas y comandos pueden evolucionar. Ante diferencias con la interfaz actual, use la documentación oficial y conserve evidencia de la decisión tomada.

Cierre. Al terminar no solo habrá construido pantallas. Habrá recorrido el trabajo de una persona desarrolladora: comprender, diseñar, modelar, implementar, probar, explicar y publicar.

La meta no es que todo salga bien al primer intento. La meta es que pueda reconocer qué sabe, localizar lo que falta y avanzar con evidencia.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack